Validation-First Modeling and Competition-Side XGBoost Exploration for the Kaggle Spaceship Titanic Benchmark

Dingrong Xue, Fan Xie, and Shijun Cheng
Artificial Intelligence Programme, Department of Computer Science
Faculty of Science and Technology
Beijing Normal-Hong Kong Baptist University (BNBU), Zhuhai, China
Team EAP_Hater@MLW; Corresponding author: Dingrong Xue
u430034083@mail.bnbu.edu.cn

***Abstract*—This paper reports our final AI3023 course project on the Kaggle *Spaceship Titanic* competition. The task is a binary classification problem on mixed-type tabular data: predict whether a passenger was transported to an alternate dimension using demographics, cabin assignments, spending behavior, and travel-group identifiers. We deliberately separate two goals that are often conflated in competition projects: obtaining a competitive leaderboard result and producing a defensible machine learning study. To do this, we benchmark five machine learning models under a repeated group-aware validation protocol, build an EDA-driven feature space, and separately analyze a competition-side XGBoost branch that produced the team's highest public score. The clean local benchmark identifies CatBoost as the strongest scientific anchor, while XGBoost offers the best speed-performance compromise and ultimately yields the best public leaderboard result through a more aggressive compact-feature branch. We also show, through shared-audit subgroup disagreement analysis, that the public-best XGBoost branch is specialized rather than uniformly superior to the cleaner CatBoost line. In the public leaderboard snapshot downloaded on May 17, 2026, our team `EAP_Hater@MLW` ranked 97th out of 2240 teams with a score of 0.81716, while our best clean single-model submission ranked 238th with a score of 0.81061. The main conclusion is that for a highly studied benchmark such as *Spaceship Titanic*, rigorous validation, ablation, and reproducibility are more important to the final analysis than optimizing solely for marginal public-leaderboard gains.**

***Index Terms*—Kaggle, tabular classification, CatBoost, XGBoost, feature engineering, cross-validation, reproducibility**

I. Introduction

The AI3023 course project requires each team to compete on Kaggle while also submitting a complete machine learning report, code archive, and presentation. This dual requirement creates a useful tension. Kaggle encourages rapid iteration against a live public leaderboard, but the course rubric rewards methodological quality, reproducibility, sound comparison, and the ability to explain why a pipeline works. On a widely used benchmark dataset such as *Spaceship Titanic*, those two incentives are related but not identical.

This distinction mattered strongly in our project. The competition already has many public notebooks, minor feature variations, and leaderboard-oriented tricks. As a result, a higher public score does not automatically imply a better scientific story. We therefore treated the project as a validation-first machine learning study, while still documenting the competition-side branch that produced the team's highest public result.

The paper makes four contributions:

1) We construct a clean, EDA-driven feature space that combines original variables with cabin, group, and spending features derived only from the official competition files.
2) We benchmark five models under one validation framework and compare both predictive quality and computational cost.
3) We define a repeated group-aware validation protocol that explicitly separates exploratory tuning from promotion decisions.
4) We reconstruct the team's highest-scoring XGBoost branch as a staged experiment, so the final public score is explained as part of our own exploration trajectory rather than as an isolated leaderboard artifact.

This framing is especially appropriate for *Spaceship Titanic*. Because the dataset is small, semantically structured, and heavily studied, local cross-validation and public leaderboard behavior do not align monotonically. Our final report therefore treats the leaderboard as one evaluation channel, not the only one.

II. Related Work

Tabular binary classification problems are often dominated by tree ensembles, especially gradient boosting systems such as XGBoost, LightGBM, and CatBoost [2]–[4]. These models can capture nonlinear interactions, tolerate heterogeneous feature distributions, and remain highly competitive when the feature engineering budget is moderate. Random Forest provides a useful bagging baseline [5], while Logistic Regression remains an important linear reference point [6].

For hyperparameter search, frameworks such as Optuna are widely used in competition workflows [7]. However, the existence of a tuning library alone is not enough to make a

TABLE I
DATASET SUMMARY AFTER INITIAL INSPECTION

Statistic	Value
Training rows	8,693
Test rows	4,277
Raw predictive columns	13
Positive class rate	50.36%
Largest missing rate	2.50%
Median age	27
Zero-spend rate	42.02%
Multi-passenger rate	44.73%
Group-homogeneous rate	87.18%

competition pipeline academically convincing. What matters more is whether the tuning target, validation regime, and promotion rule are stable and clearly documented. More broadly, leaderboard methodology itself has been studied as a reliability problem: Blum and Hardt show that repeated interaction with a public leaderboard can distort model comparison if public score is treated as the only selection signal [8]. That perspective is directly relevant to our course setting.

Open Kaggle notebooks are also a major source of practical ideas for this competition. Representative references include Cortinhas' complete guide and Arunklenin's EDA plus feature engineering notebook [9], [10]. Across these references, several ideas recur consistently: cabin decomposition is useful, passenger-group structure matters, and aggregated spending is more stable than treating every ancillary-spend variable independently. We used such public material as background survey and idea generation, but not as a substitute for our own validation and ablation.

III. Dataset and Exploratory Analysis

A. Dataset Characteristics

The official competition dataset contains 8,693 labeled training rows and 4,277 unlabeled test rows [1]. The raw schema has 13 predictive columns spanning demographics, trip destination, cryosleep status, cabin identifier, five spending variables, and passenger name. The target variable `Transported` is nearly balanced: the positive-class rate in the training data is 50.36%. Missingness is mild, with the largest raw-feature missing rate equal to 2.50%.

The data is not i.i.d. in a naive sense. Each `PassengerId` has the form `gggg_pp`, where the prefix `gggg` identifies passengers traveling together. In our own summary, 44.73% of rows belong to multi-passenger groups, and travel-group structure is therefore too strong to ignore during validation.

Table I summarizes the core statistics used throughout the paper.

B. Exploratory Data Analysis

Our EDA was designed to support model choices, not merely to decorate the report. Figure 1 confirms two global properties: missingness is modest but non-zero, and the target is balanced. This justifies accuracy as the main competition

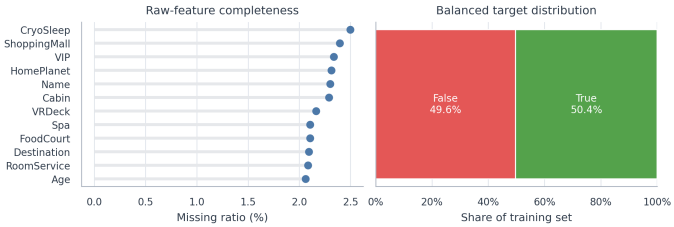

Fig. 1. Data completeness and target balance. The dataset is not sparse, but careful imputation remains necessary.

metric while still motivating supporting metrics such as F1 and ROC-AUC for local comparison.

The more interesting patterns are shown in Figure 2. Age contributes signal in a nonlinear way, especially when separating children from adults. Spending is much more discriminative: transported passengers are far more concentrated near zero ancillary spend, which is consistent with the domain interpretation of cryosleep. The categorical views show large rate differences across home planets and cabin decks, which motivates both category-aware models and deck-based engineered features.

Figure 3 adds two more design cues. First, solo travelers are the single largest segment, but multi-passenger groups still occupy a large fraction of the dataset; this supports group-aware validation and explicit group features. Second, the spend variables are correlated with total spend but not perfectly redundant, which argues for keeping both aggregate and component-level information.

IV. Methodology

A. Preprocessing and Feature Engineering

Our main preprocessing rule is simple: every engineered feature must be reproducible from the official competition files alone and easy to defend during presentation. The final feature space consists of five families:

1) Original demographics and travel descriptors: `HomePlanet`, `Destination`, `Age`, `VIP`, and `CryoSleep`.
2) Cabin-derived features: `Deck`, `CabinNum`, `Side`, and deck-side combinations.
3) Passenger-group-derived features: `PassengerGroup`, `PassengerNo`, `GroupSize`, and `IsAlone`.
4) Spend-derived features: `TotalSpend`, zero-spend flags, spend-composition ratios, and log-transformed spend.
5) Rule-consistency features: especially cryosleep-spend consistency indicators.

Table II summarizes these families and the intuition behind them.

B. Validation Protocol

The most important methodological choice in this project is the validation design. Because group members often share outcomes, row-level random validation leaks relational structure into the validation folds. We therefore adopted repeated

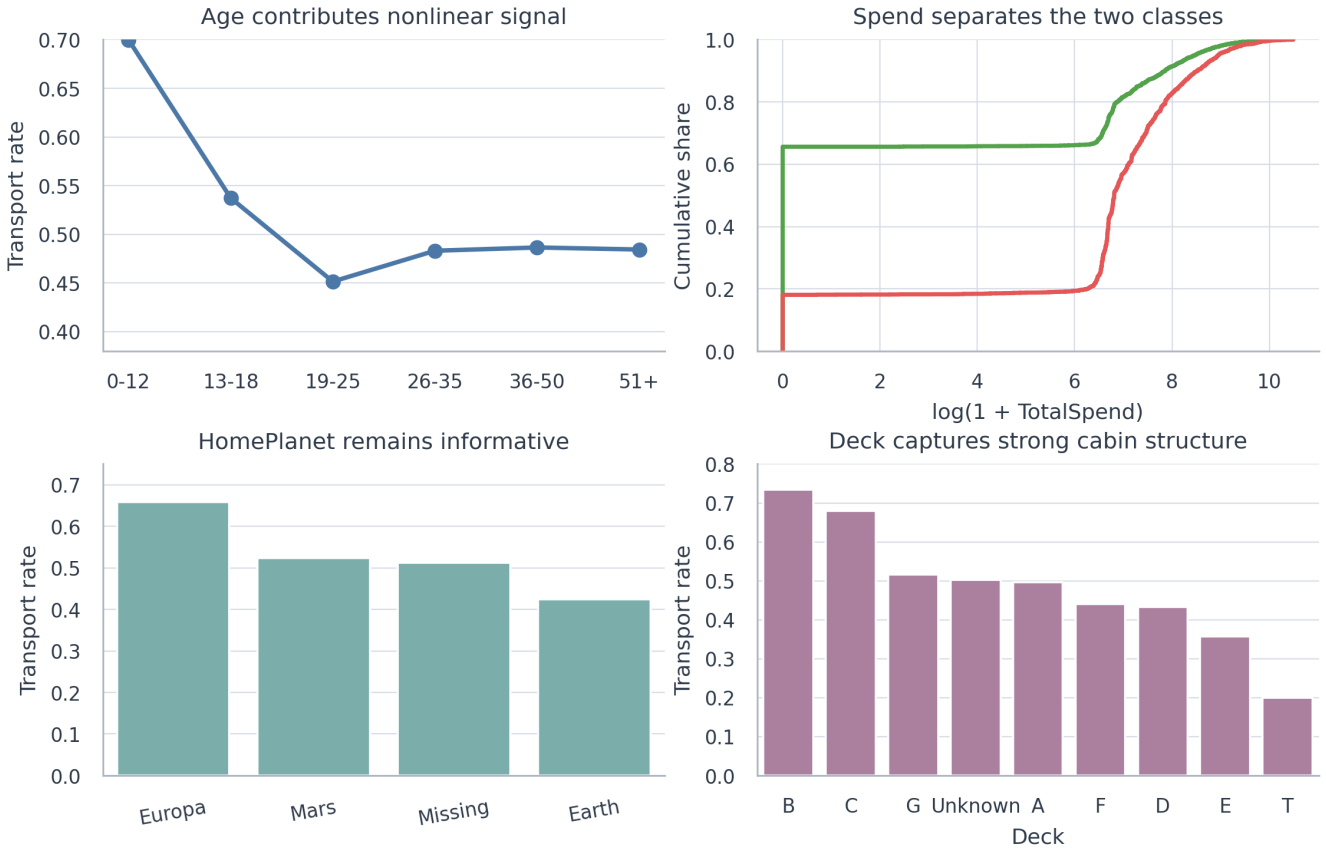

Fig. 2. Core EDA findings. Spending, home planet, and deck have clear predictive value, while age contributes weaker but still meaningful nonlinear structure.

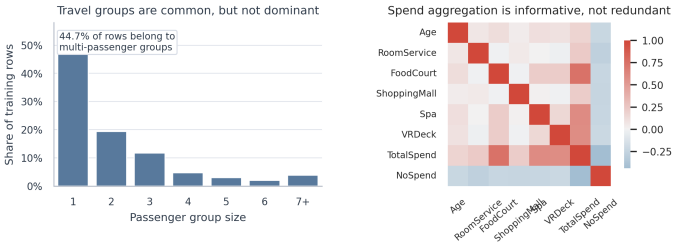

Fig. 3. Passenger-group structure and numeric correlation. These patterns motivate group-aware validation and spend aggregation instead of raw-column-only modeling.

`StratifiedGroupKFold` grouped by the `PassengerId` prefix.

We also separated seeds into two roles:

- Search seeds {7, 21, 42, 87, 123} for exploratory tuning and feature experiments.
- Audit seeds {3, 11, 17, 29, 57, 68, 99, 131} for promotion decisions only.

Our promotion rule was conservative: a candidate should improve out-of-fold accuracy by at least 0.001 on both seed sets, while keeping the audit threshold fixed at 0.5. This rule prevented us from promoting many attractive but unstable changes.

TABLE II
ENGINEERED FEATURE FAMILIES

Family	Rationale and examples
Original demographics	`HomePlanet`, `Destination`, `Age`, `VIP`, `CryoSleep`. These preserve the official schema.
Cabin-derived	`Deck`, `CabinNum`, `Side`, and `DeckSide`. These encode spatial structure inside the ship.
Passenger-group-derived	`PassengerGroup`, `PassengerNo`, `GroupSize`, and `IsAlone`. These model travel-group behavior explicitly.
Spend-derived	`TotalSpend`, `NoSpend`, spend ratios, and `LogTotalSpend`. These summarize luxury-service behavior more cleanly.
Rule-consistency	Cryosleep-spend consistency flags. These encode plausible behavioral constraints.

C. Model Families and Tuning Strategy

The course requires at least four models, so we benchmarked five: Logistic Regression, Random Forest, LightGBM, XGBoost, and CatBoost. Logistic Regression and Random Forest serve as linear and bagging baselines. LightGBM, XGBoost, and CatBoost represent the dominant boosting family for tabular competitions.

Tuning strategy depended on the role of each model. The validation-first benchmark used a common feature space and repeated group-aware validation so that models remained comparable. LightGBM and CatBoost were tuned locally around

TABLE III
COMPETITION-SIDE XGBOOST BRANCH CONFIGURATION

Component	Setting
Categorical harmonization	Train and test concatenated before categorical imputation and one-hot alignment
Imputation logic	Mean imputation for numeric columns, most-frequent imputation for compact categorical set, plus room-guided filling for `VIP`, `Cabin`, `HomePlanet`, and `Destination`
Compact branch features	27 encoded features before pruning; 15 after pruning
Outlier diagnostic	`IsolationForest` with contamination = 0.003 was evaluated during branch reconstruction but not used in the final submitted training matrix
Class balancing	`KMeansSMOTE`; training rows increased to 8,762
Final XGBoost setting	seed 36086; `max_depth`=5, `n_estimators`=950, `learning_rate`=0.0475, `subsample`=0.96, `colsample_bytree`=0.93, `reg_alpha`=4.582, `reg_lambda`=3.06

strong baselines. XGBoost appeared in two different roles:

1) as part of the clean unified benchmark;
2) as a separate competition-side branch designed to maximize public leaderboard performance.

The competition-side XGBoost branch intentionally relaxes some of the cleanliness constraints of the main benchmark. It concatenates train and test during categorical harmonization, performs room-guided imputation, reduces the encoded feature space from 27 to 15 dimensions through manual pruning, balances the training matrix using `KMeansSMOTE`, and finally trains a regularized XGBoost model. The final public-best setting used random seed 36086, `max_depth`=5, `n_estimators`=950, `learning_rate`=0.0475, `subsample`=0.96, `colsample_bytree`=0.93, `reg_alpha`=4.582, and `reg_lambda`=3.06. We report this branch separately because it was optimized for competition performance, not because it is the cleanest scientific baseline.

D. Experimental Controls and Reproducibility

To keep the report defensible, we maintained several controls across the validation-first benchmark. All unified model comparisons used the same feature families, the same group-aware split design, the same fixed threshold on audit seeds, and no external prediction files. This makes the benchmark table interpretable as a true model-family comparison rather than a comparison between unrelated pipelines.

We also maintained separate tuning evidence for the clean benchmark models. The LightGBM random-search winner improved audit accuracy by +0.0031 over the default baseline, while the best CatBoost sweep improved audit accuracy by +0.0023. We report these gains explicitly later because the course brief requires hyperparameter tuning to be demonstrated rather than merely asserted.

We deliberately isolated the competition-side XGBoost branch from this benchmark. Its purpose was different: maximize public leaderboard score within the official competition rules. By separating the two tracks, we avoided a common failure mode in course reports where a leaderboard-oriented branch silently becomes the "best model" even when it is no longer the cleanest scientific baseline.

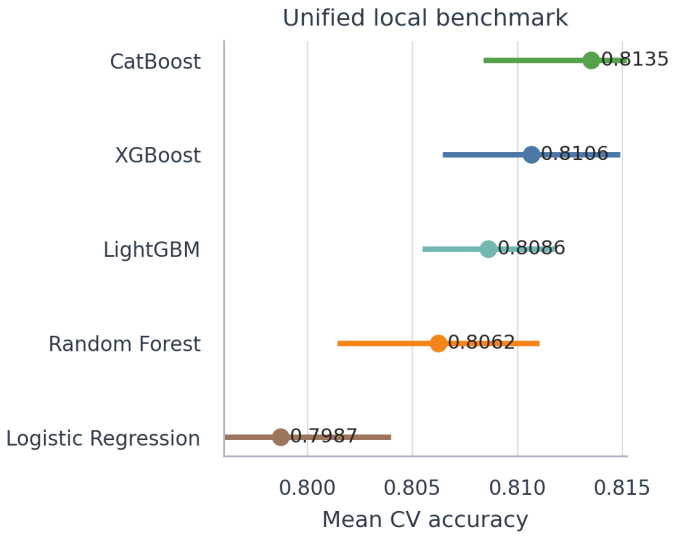

Fig. 4. Mean CV accuracy under the unified benchmark. Boosting clearly dominates the task.

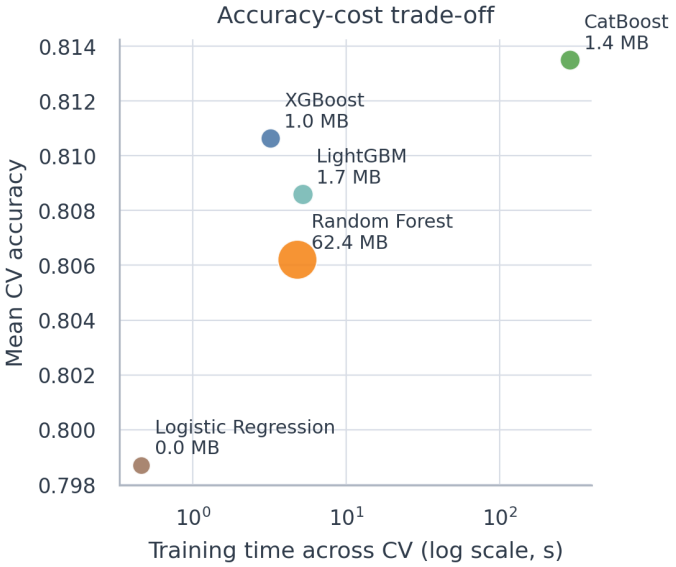

Fig. 5. Accuracy-cost trade-off. CatBoost is strongest locally, while XGBoost is the most attractive speed-performance compromise.

V. EXPERIMENTAL STUDY AND RESULTS

A. Unified Local Benchmark

Table IV reports mean CV accuracy, OOF F1, OOF AUC, training time, inference cost, and artifact size under the same benchmark scaffold. This satisfies the course requirement that models be compared by both performance and practicality.

The ranking is visually summarized in Figure 4. CatBoost is the strongest local model, XGBoost is only slightly weaker while being dramatically faster to train, and the linear baseline underfits the task.

TABLE IV
UNIFIED LOCAL BENCHMARK UNDER THE SAME FEATURE SET AND VALIDATION REGIME

Model	Mean CV Acc.	OOF F1	OOF AUC	CV Fit Time (s)	Test Infer. (ms/sample)	Artifact Size (MB)
CatBoost	0.8135	0.8143	0.9069	288.32	0.0230	1.37
XGBoost	0.8106	0.8103	0.9034	3.19	0.0061	1.04
LightGBM	0.8086	0.8099	0.9012	5.20	0.0089	1.65
Random Forest	0.8062	0.8013	0.8930	4.79	0.0350	62.40
Logistic Regression	0.7987	0.8047	0.8887	0.46	0.0040	0.01

B. Why the Models Behave Differently

The benchmark is informative not only for ranking models, but also for explaining fit to the data. Logistic Regression is weakest because the task contains interacting nonlinear effects: cryosleep interacts with spending, passenger groups interact with cabin information, and cabin identifiers only become informative after decomposition. Random Forest improves on the linear baseline by modeling interactions, but it pays a large memory cost without matching the boosted methods.

LightGBM and XGBoost fit the task more naturally. Both exploit nonlinear interactions and support fast iteration on encoded tabular features. CatBoost performs best locally because the dataset contains exactly the kind of mixed categorical and numerical structure for which CatBoost is especially strong. This aligns with our EDA findings: home planet, deck, side, group, and spending all interact in ways richer than a purely numeric benchmark, but still well within the regime where classical boosted trees excel.

Table V summarizes these qualitative fit arguments.

C. Interpretability of the Clean CatBoost Anchor

The clean CatBoost anchor is also the easiest model to interpret. Figure 6 shows the top feature importances extracted from the validation-first CatBoost benchmark. `TotalSpend`, `Deck`, `HomePlanet`, and `PassengerGroup` dominate the ranking, which is consistent with the strongest EDA signals. The same figure also explains why the clean CatBoost pipeline is such a good report centerpiece: the leading signals are semantically meaningful and align with domain intuition instead of depending on opaque post-processing.

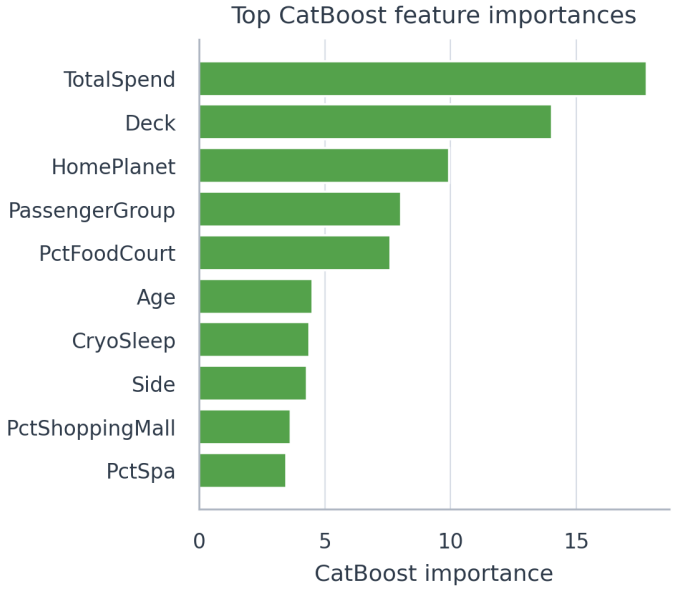

Fig. 6. Top feature importances from the clean CatBoost anchor. The strongest signals align well with the EDA findings.

D. Feature-Family Ablation

To test whether every plausible feature family was actually helpful, we ran a cumulative ablation study on the clean CatBoost line under the same group-aware validation protocol. Figure 7 summarizes the result.

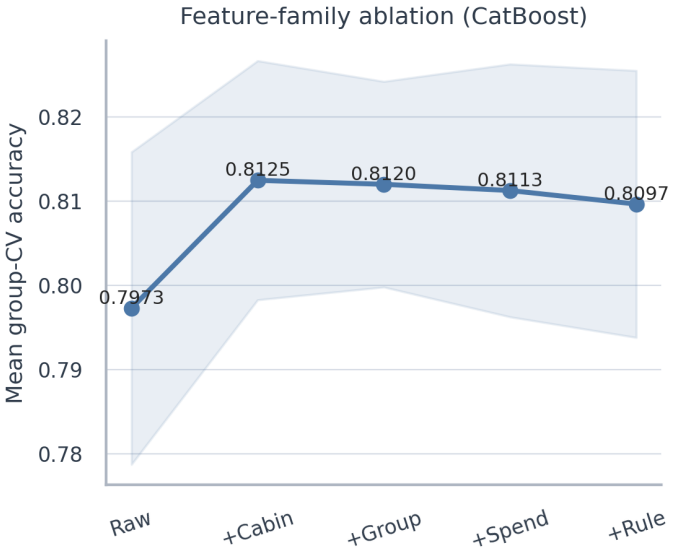

Fig. 7. Cumulative feature-family ablation on the clean CatBoost line. Cabin decomposition produced the largest single improvement.

The pattern is more informative than a simple monotonic feature-accumulation narrative. Raw official features alone reached only 0.7973. Adding cabin decomposition raised group-aware CV to 0.8125, which was the largest jump in the entire study. Adding group features kept performance near the same level, while spend-summary and rule-consistency features provided no further gain under this fixed CatBoost setting and slightly reduced the final mean. This does not mean that group or spend information is useless. Rather, under this particular CatBoost configuration, those later families are partly redundant with the already strong cabin-enhanced core representation instead of being additively beneficial. That

TABLE V
QUALITATIVE SUITABILITY OF EACH BENCHMARKED MODEL FOR THIS DATASET

Model	Main strengths on this task	Main weaknesses on this task	Overall suitability
Logistic Regression	Very fast, interpretable, and useful as a sanity-check baseline.	Cannot capture nonlinear cryosleep-spend-group interactions compactly.	Low. Good baseline, weak final model.
Random Forest	Flexible nonlinear baseline with few distributional assumptions.	Large artifact size and weaker score than boosting under the same feature space.	Moderate. Reasonable baseline, not the best trade-off here.
LightGBM	Fast training and strong performance on sparse encoded features.	Small local gains proved unstable under wider audit checks.	High. Strong clean benchmark model.
XGBoost	Best speed-performance compromise and highly competitive competition model.	Sensitive to preprocessing and feature-pruning choices.	High. Strong practical model, especially for the competition-side branch.
CatBoost	Best local accuracy and very natural fit for mixed-type tabular data.	Slower to train; not the top public-score branch.	Very high. Best scientific anchor for the final report.

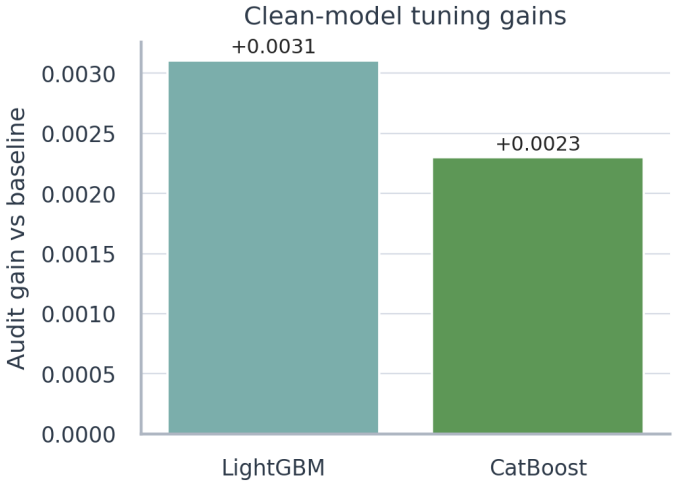

Fig. 8. Audited tuning gains for the clean LightGBM and CatBoost search tracks retained for the final analysis.

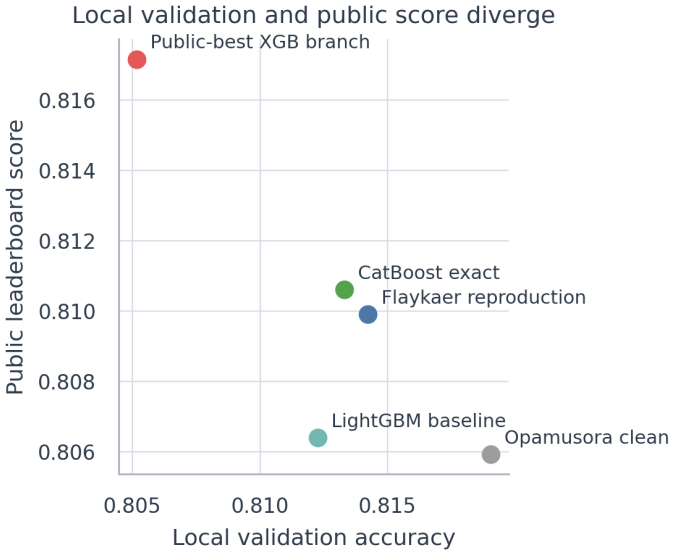

Fig. 9. Local validation does not monotonically predict public leaderboard score. This is why the report emphasizes validation design and ablation rather than public-score optimization alone.

finding supports our conservative promotion rule and shows that the final report is not based on indiscriminate feature accumulation.

E. Tuning Evidence on Clean Models

Figure 8 summarizes the audited gains from the two clean-model tuning tracks retained for the final analysis.

The LightGBM search track improved audit accuracy by +0.0031 over its default baseline through random search with independent audit seeds. The CatBoost sweep improved audit accuracy by +0.0023 over the clean baseline through a smaller local parameter sweep. These deltas are meaningful enough to satisfy the course requirement for hyperparameter tuning, while still remaining inside a report-standard workflow.

Table VI makes the tuning path more concrete. The winning LightGBM candidate became smaller in leaf capacity, deeper in explicit tree depth, and more conservative through stronger regularization and larger minimum leaf size. The winning CatBoost candidate moved in a similar direction: slightly deeper trees, stronger L_2 regularization, modest subsampling, and less aggressive feature sampling noise. In other words, the preserved search evidence points to disciplined regularization and capacity control rather than to exotic hyperparameter tricks.

F. Validation Alignment versus Public Leaderboard

One of the central findings of the project is that local validation and public leaderboard behavior do not align monotonically. Figure 9 shows this directly. The most striking counterexample is a high-CV CatBoost candidate, which reached 0.81905 under local group-aware validation but only 0.80593 publicly. The inverse pattern is the team's public-best XGBoost branch, which achieved only 0.80516 under our strict group-aware audit but scored 0.81716 on the public leaderboard.

This mismatch changed our workflow. First, we stopped trusting small local improvements unless they survived the audit seeds. Second, we separated the best clean single model from the best public submission instead of forcing one narrative to explain both. The best clean single model is the CatBoost pipeline, which scored 0.81061 publicly and is straightforward to explain. The best public result belongs to a different XGBoost branch with a different objective.

Table VII summarizes the candidate lines that mattered most to this conclusion. The table is useful because it separates three different concepts that are easy to blur together in

TABLE VI
KEY HYPERPARAMETER CHANGES RETAINED FROM THE CLEAN TUNING RECORDS

Model	Baseline setting	Tuned winner setting	Audit gain
LightGBM	500 trees, 31 leaves, unlimited depth, `subsample`=0.90, `colsample`=0.80, `reg_alpha`=0.10, `min_child_samples`=25	1000 trees, 15 leaves, depth 6, full row/column sampling, `reg_alpha`=0.70, `min_child_samples`=35	+0.0031
CatBoost	depth 6, `l2_leaf_reg`=3, `subsample`=0.80, `rsm`=1.00, `random_strength`=1, `min_data_in_leaf`=1	depth 7, `l2_leaf_reg`=7, `subsample`=0.88, `rsm`=0.85, `random_strength`=0.5, `min_data_in_leaf`=8	+0.0023

a Kaggle report: the strongest clean scientific anchor, the strongest local-CV counterexample, and the strongest public-leaderboard branch.

G. The XGBoost Exploration Branch

The highest public score in the team history did not appear from a single isolated submission. It emerged through an XGBoost-centered exploration line. Figure 10 shows this trajectory. An early single XGBoost submission reached 0.79378 on April 1, 2026, a revised single-model version reached 0.80383 on April 10, 2026, XGBoost-assisted stacking reached 0.81084 on April 16, 2026, and the final public-best XGBoost branch reached 0.81716 on April 26, 2026.

This distinction is important because we do not claim that the final XGBoost parameters are a unique local-CV optimum. Instead, we claim something narrower and more reproducible: the public-best branch is backed by visible experimental provenance. The parameter set began from an Optuna-guided compact XGBoost branch; the preprocessing, manual feature pruning, and `KMeansSMOTE` design were then retained as fixed branch decisions; and a focused competition-side search adjusted the tree count, learning rate, and random seed. The final public-best setting used 950 trees, a learning rate of 0.0475, and seed 36086, while retaining the earlier regularization and sampling choices. A local neighborhood search further showed that the final branch lies inside a competitive plateau rather than far away from reasonable alternatives. Thus, the public-best submission should be read as an evidence-backed competition artifact, not as an unsupported parameter choice.

To understand why the final branch worked, we reconstructed it in stages and measured how its local behavior changed. Table VIII reports the result. The compact preprocessing branch itself only reached 0.80445 under 10-fold row-level reconstruction CV. The `IsolationForest` diagnostic made this reconstruction metric worse when treated as a filtering step, while feature pruning raised it to 0.80755. Re-evaluated under our stricter 5-fold `StratifiedGroupKFold` audit, the same branch scored 0.80516.

Table IX makes the public-side evolution explicit. The rejected tuned XGBoost variant on April 10, 2026 is especially informative: parameter changes alone did not help, and the decisive improvement came only after the project explored XGBoost as part of a broader competition branch rather than as a direct single-model replacement.

We also ran a focused local neighborhood search around the branch parameters to test whether the public-best configuration was at least locally competitive under standard validation. Figure 11 shows the result. Row-level CV slightly preferred a lower learning rate with more trees, while the group-aware audit slightly preferred a leaner sampling setting. However, all tested variants stayed within a narrow accuracy band, and none emerged as a dominant local winner under both views simultaneously.

This is an important result for the logic of the paper. It means the final public-best XGBoost branch should not be described as the unique locally optimal model. A more precise interpretation is that it lies inside a competitive parameter region, and its public superiority depends on leaderboard interaction in a way that local CV does not fully explain. This framing is more defensible than claiming a tuning result that the local evidence does not support.

Two points follow from this table. First, the final branch is not an isolated parameter point; it is the endpoint of a longer XGBoost exploration line in our own submission history. Second, the branch is best interpreted as a competition-side artifact rather than as our strongest scientific baseline. Its public success is real, but the strongest methodological claim of the paper still comes from the validation-first benchmark.

H. Where the Two Branches Disagree

The previous subsection explains why the public-best XGBoost branch should not be described as a universal local optimum. A complementary question is whether it at least behaves like a uniformly stronger classifier than the cleaner CatBoost line. Under one shared 5-fold `StratifiedGroupKFold` audit, the answer is no. The two branches disagree on 8.49% of training rows, and the cleaner CatBoost line keeps a 0.37-point accuracy advantage overall under that shared audit.

Figure 12 shows where that disagreement concentrates. At the cabin level, the two branches diverge most strongly on decks E and G. At the behavioral level, disagreement is highest in cryosleep-spend edge cases, especially passengers with missing `CryoSleep` but zero spend and passengers with `CryoSleep=False` but zero spend. These are precisely the kinds of ambiguous cases where heuristic competition-side preprocessing can change predictions substantially. Importantly, XGBoost is not uniformly worse either: it is slightly better on some narrower subsets such as Deck A and Deck E. The right interpretation is therefore a specialization story. The compact XGBoost branch changes a small but meaningful subset of decisions, some of which help publicly, but it does not dominate the cleaner CatBoost line across the board.

TABLE VII
REPRESENTATIVE CANDIDATE LINES AND WHAT THEY TAUGHT US

Candidate line	Local metric	Public score	Role in the project	Main lesson
LightGBM baseline	0.81226	0.80640	Early validation-first baseline	Useful baseline for promotion decisions, but not strong enough as a final competition submission.
Clean CatBoost single model	0.81330	0.81061	Best clean single-model anchor	The most defensible report centerpiece: strong score, simple preprocessing, and high interpretability.
Clean CatBoost reproduction	0.81422	0.80991	Transparent clean reproduction	A respectable public reproduction that still failed to beat the clean anchor.
High-CV CatBoost candidate	0.81905	0.80593	High-CV counterexample	Strongest evidence that local group-aware CV alone is not a dependable public-score ranker near the top of this competition.
Public-best XGB branch	0.80516	0.81716	Best leaderboard branch	Highest public result, but not the strongest scientific baseline; must be explained as a separate competition-side line.

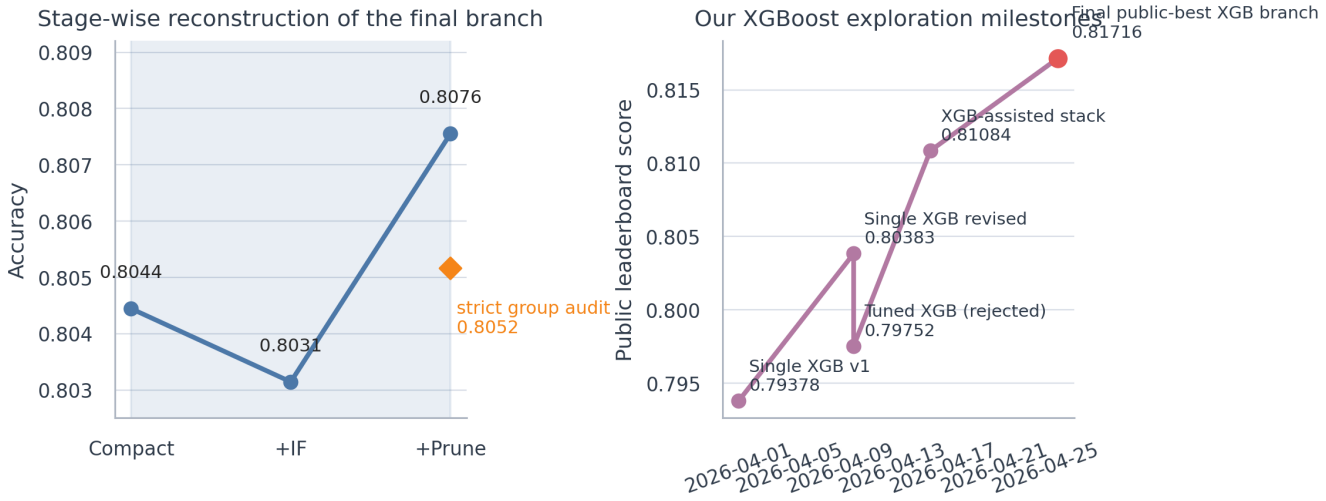

Fig. 10. Left: stage-wise reconstruction of the final XGBoost branch. Right: the public-score trajectory of our XGBoost-centered exploration path from April 1, 2026 to April 26, 2026.

TABLE VIII
STAGE-WISE RECONSTRUCTION OF THE FINAL XGBOOST BRANCH

Stage	Validation view	Accuracy	Std. dev.	Interpretation
Compact preprocessing branch	10-fold row-level reconstruction CV	0.80445	0.01417	A workable but not yet competitive compact XGBoost baseline.
+ IsolationForest diagnostic	10-fold row-level reconstruction CV	0.80314	0.01590	Outlier filtering did not help locally and was not used in the final submitted training matrix.
+ Feature pruning	10-fold row-level reconstruction CV	0.80755	0.01542	Manual pruning recovered a clearer gain and created the final compact branch used for the public-best submission.
Strict group-aware audit	5-fold StratifiedGroupKFold	0.80516	0.00490	Under the stricter report-standard audit, the branch remains weaker than the clean CatBoost anchor despite its public success.

TABLE IX
MILESTONES IN THE XGBOOST EXPLORATION BRANCH

Date	Milestone	Public score	Why it mattered
2026-04-01	Single XGB v1	0.79378	Established an early XGBoost baseline and showed that a straightforward single-model configuration was not yet competitive.
2026-04-10	Single XGB revised	0.80383	Confirmed that XGBoost could improve materially with better preprocessing, even before branch-level redesign.
2026-04-10	Tuned XGB (rejected)	0.79752	Demonstrated that naive tuning did not help and that parameter changes alone were insufficient.
2026-04-16	XGB-assisted stack	0.81084	Showed that XGBoost became useful as a controlled component inside a stronger blended system.
2026-04-26	Final public-best XGB branch	0.81716	Produced the highest public score in the team history and finalized the competition-side narrative.

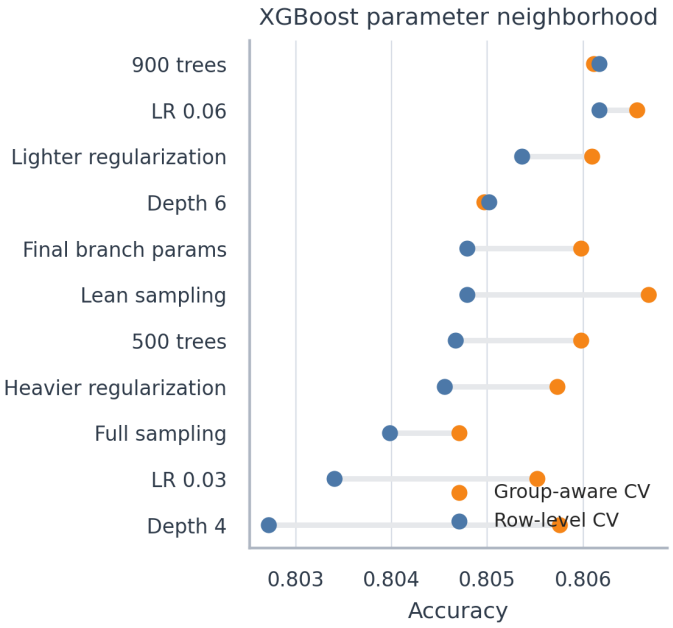

Fig. 11. Focused local neighborhood search around the final XGBoost branch. The public-best configuration sits inside a competitive local plateau rather than as a dominant local optimum.

I. Final Kaggle Ranking and Submission History

The public leaderboard snapshot downloaded on May 17, 2026 places our team `EAP_Hater@MLW` at Rank 97 out of 2240 teams with a score of 0.81716. This corresponds to approximately the top 4.3% of ranked teams in the downloaded snapshot. The best clean single-model submission reached 0.81061 and Rank 238.

Figure 13 shows the overall public-score trajectory, while Figure 14 situates our score inside the distribution of all public leaderboard scores. For course-compliance purposes, Figure 15 also includes a curated snapshot of representative Kaggle submission milestones retrieved through the Kaggle API on May 17, 2026. Semantic labels are used for readability, while the raw submission history is preserved in the exported tables.

VI. Discussion

Three lessons are worth emphasizing.

First, *Spaceship Titanic* is not a good setting for uncritical public-score optimization if the goal is an academically strong report. Because the dataset is small and highly exposed, public-score changes can reflect split-specific behavior rather than reliable generalization. This concern is consistent with prior work on leaderboard reliability, which treats repeated public-score interaction as a genuine evaluation problem rather than a harmless convenience [8].

Second, the strongest clean single-model pipeline and the strongest public-scoring pipeline are not the same object. The clean CatBoost single-model line is the best narrative anchor for a classroom report because it is transparent, stable, and easy to justify. The public-best XGBoost branch is the correct competition artifact because it demonstrably delivered the highest team score. Treating these two roles separately resolves the main tension in the project.

Third, the most valuable scientific contribution of the work is the validation framework itself. The repeated group-aware design, the split between search seeds and audit seeds, and the refusal to promote sub-thousandth gains without wider checking all made the project more trustworthy. On this dataset, that discipline is more important than small public-score fluctuations.

VII. Threats to Validity

This project has four main validity threats.

First, the public leaderboard is only a partial view of the hidden Kaggle test set. Even a real public gain may not transfer perfectly to the private split. This is why the paper does not equate public-best score with universal model superiority.

Second, the competition-side XGBoost branch uses choices that are more acceptable in a competition than in a clean scientific benchmark, especially train-plus-test harmonization and manual encoded-feature pruning. We therefore describe it explicitly as a competition-side branch rather than as the main scientific conclusion.

Third, the final public-best XGBoost branch includes a seed-specific competition-side selection step. We therefore report it as the final submitted branch rather than as a generally optimal hyperparameter setting.

Fourth, the whole study is still conducted inside one competition ecosystem. Even though our validation discipline is stronger than a naive single split, the conclusions remain about *Spaceship Titanic* specifically and should be read as a careful case study in tabular competition methodology.

VIII. Future Work and Conclusion

If we were extending the project beyond the course deadline, the next priorities would not be another leaderboard-oriented adjustment. More meaningful directions would be nested group-aware hyperparameter tuning, fold-wise error analysis for group-consistency failures, probability calibration, and stability checks on feature-importance explanations. A more detailed probability-level analysis of the CatBoost–XGBoost disagreement regions would also strengthen the interpretability story.

In conclusion, the project achieved both of its goals, but through different model lines. On the competition side, the team reached 0.81716 and Rank 97/2240 in the public leaderboard snapshot dated May 17, 2026. On the methodological side, the report established a reproducible validation-first workflow, benchmarked five models, documented computational trade-offs, and showed that local validation and public leaderboard behavior can diverge sharply in both directions. For a widely studied Kaggle dataset such as *Spaceship Titanic*, that final lesson is arguably the most valuable one: robust conclusions come from disciplined validation and careful ablation, not from equating a public leaderboard improvement with scientific truth.

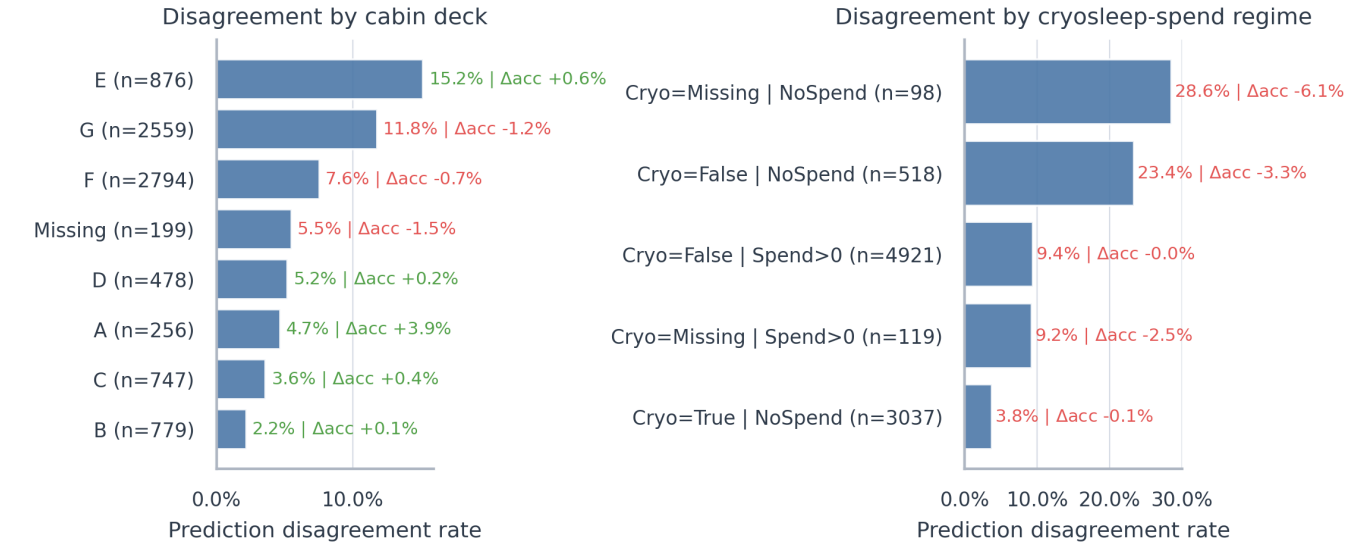

Fig. 12. Shared-audit disagreement between the clean CatBoost line and the compact XGBoost branch. Text annotations report subgroup disagreement rate and local accuracy gap (XGBoost minus CatBoost).

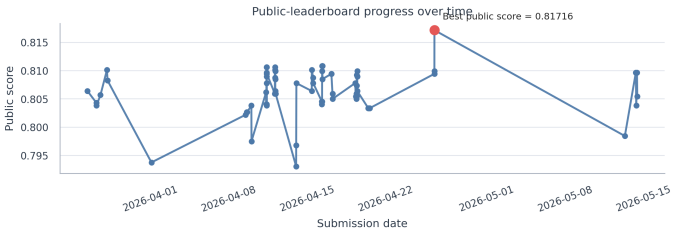

Fig. 13. Public leaderboard progress over time. The best visible submission remains the XGBoost branch submitted on April 26, 2026.

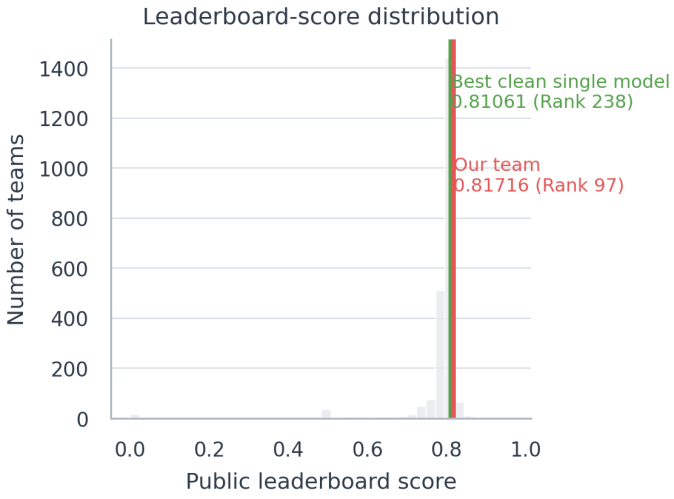

Fig. 14. Leaderboard-score distribution in the public snapshot downloaded on May 17, 2026. The team result and the best clean single-model result are highlighted.

IX. Code Availability and Contribution Statement

A. Code Availability

The final IEEE paper, generated figures, tables, reproducible notebook, and code are organized in the submitted repository: https://github.com/Dylan4X/spaceship-titanic-mlworkshop

B. Contribution Statement

Dingrong Xue contributed 34% of the project work, including Kaggle leaderboard experimentation, validation design, model benchmarking, final report writing, experimental refinement, and reproducible code preparation. Fan Xie contributed 33%, including Kaggle leaderboard experimentation, competition-side XGBoost exploration, hyperparameter search, result discussion, and presentation slide preparation. Shijun Cheng contributed 33%, including exploratory data analysis, feature engineering support, interpretation of dataset patterns, visualization preparation, and presentation slide preparation. All members participated in final project review.

Appendix A
Requirement Traceability

Table X maps the course report requirements to concrete evidence in this paper.

Appendix B
Milestone Submission Record

Table XI records the milestone submissions that matter most to the final narrative.

This appendix is included for two practical reasons. First, the course requires evidence of the final Kaggle workflow rather than only the final score. Second, the main body of the paper repeatedly distinguishes between clean scientific anchors and competition-side artifacts; the milestone list makes that distinction concrete in submission-history terms.

Read together with Figure 13, this appendix clarifies the project chronology. The April 26, 2026 XGBoost branch remained the best visible public result through the final snapshot on May 17, 2026, while the cleaner CatBoost line remained

Representative Kaggle submission milestones retrieved on 2026-05-17

Milestone	Submission time	Public score	Role
Clean CatBoost exact line	2026-04-11 09:36	0.81061	Clean anchor
Flaykaer clean reproduction	2026-04-12 16:09	0.80991	Clean reproduction
High-CV CatBoost counterexample	2026-04-12 17:34	0.80593	Validation counterexample
XGB-assisted stack	2026-04-16 11:57	0.81084	Competition-side hybrid
Final public-best XGB branch	2026-04-26 08:10	0.81716	Public-best branch
Late blend attempt	2026-05-14 09:37	0.80967	Late check

Fig. 15. Representative Kaggle submission milestones retrieved on May 17, 2026. Semantic labels are used in place of raw file names; the best visible score is the final public-best XGBoost branch at 0.81716.

TABLE X
TRACEABILITY FROM COURSE REQUIREMENTS TO EVIDENCE IN THE FINAL PAPER

Course requirement	Evidence in this paper
Abstract	Abstract section on page 1 summarizes the task, methods, and final outcomes.
Introduction	Section I explains the competition setting and the distinction between leaderboard outcome and scientific claim.
Related work / existing techniques	Section II discusses boosting models, validation practice, and representative Kaggle guides.
Methodology	Section IV documents preprocessing, feature engineering, validation design, and the competition-side XGBoost branch.
Experimental study and results analysis	Section V contains the unified benchmark, CV-public alignment analysis, XGBoost branch reconstruction, and final ranking discussion.
Future work and conclusion	Section VIII provides both next steps and final conclusions.
References	The bibliography cites the competition page, model papers, Optuna, and representative Kaggle notebooks.
Contribution statement	Section IX contains a member-level contribution statement with named authors and workload percentages.
Final Kaggle ranking	Section V-E and Figures 13–14 report Rank 97/2240 with public score 0.81716 in the May 17, 2026 snapshot.
GitHub link of the code	Section IX-A provides the repository URL field for the submitted code.
Screenshot(s) of Kaggle submission log(s)	Figure 15 provides a cleaned submission-log snapshot with representative milestones retrieved on May 17, 2026.

TABLE XI
MILESTONE SUBMISSION RECORD USED IN THE FINAL ANALYSIS

Date	Milestone label	Public score	Pipeline type	Why it matters
2026-04-11	Clean CatBoost single model	0.81061	Clean single model	Best clean single-model anchor used for most of the report narrative.
2026-04-12	Clean CatBoost reproduction	0.80991	Clean reproduction	Transparent group-aware CatBoost reproduction that remained interpretable but did not beat the clean anchor.
2026-04-12	High-CV CatBoost candidate	0.80593	High-CV clean candidate	Strong local-CV counterexample showing that public and local metrics can diverge sharply.
2026-04-16	XGB-assisted stack	0.81084	Competition-side hybrid	Demonstrates that XGBoost became useful before the final public-best branch.
2026-04-26	Final public-best XGB branch	0.81716	Competition-side compact branch	Highest public score in the team history and the endpoint of the XGBoost exploration path.
2026-05-14	Late blend attempt	0.80967	Late ensemble check	Confirms that later submissions did not surpass the April 26, 2026 public best.

the report's main scientific anchor. That dual interpretation is central to the paper and is therefore documented explicitly here.

REFERENCES

[1] Kaggle, "Spaceship Titanic," online competition page. Available: https://www.kaggle.com/competitions/spaceship-titanic/overview

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017.

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.

[8] A. Blum and M. Hardt, "The Ladder: A reliable leaderboard for machine learning competitions," in *Proc. 32nd Int. Conf. Machine Learning*, 2015, pp. 1006–1014.
[9] S. Cortinhas, "Spaceship Titanic: A complete guide," Kaggle notebook, 2022. Available: https://www.kaggle.com/code/samuelcortinhas/spaceship-titanic-a-complete-guide
[10] Arunklenin, "Space Titanic — EDA — Advanced Feature Engineering," Kaggle notebook, 2024. Available: https://www.kaggle.com/code/arunklenin/space-titanic-eda-advanced-feature-engineering